

Illegal Parking and Wrong-Side Driving Detection System

Technical Documentation & Logic Specification

Zone-based, motion-fused detection of two distinct roadside anomalies from a single per-vehicle decision pipeline: stationary vehicles dwelling in a marked no-parking zone beyond a class-dependent time threshold, and vehicles moving against the locally defined direction of travel.

1 Overview

The **Illegal Parking and Wrong-Side Driving (IPWSD)** module evaluates each tracked vehicle, frame by frame, using a branched decision pipeline driven by the vehicle’s instantaneous speed, its position relative to pre-segmented zones, and the sign of its motion along the locally defined direction axis. The overall flowchart is illustrated in Figure 1.

The module receives a pre-processed, enhanced CCTV frame from the image enhancement stage. From this single input, two independent anomaly checks are branched: a *stationary-vehicle* branch that tests for illegal parking, and a *moving-vehicle* branch that tests for wrong-side driving. The split is governed by a single decision: whether the vehicle’s speed is zero.

If the vehicle is stationary (speed = 0), the module checks whether the vehicle’s centroid lies inside a pre-marked no-parking zone, and if so whether it has dwelled there for longer than a class-dependent offset time t_{offset} (nominally 10 s for cars, 60 s for buses, to allow for brief, legitimate halts such as passenger drop-off). Exceeding this threshold inside the zone results in an **Illegal Parking** classification; otherwise the vehicle is marked **No Anomaly**.

If the vehicle is moving (speed $\neq 0$), the module instead checks the sign relationship between the vehicle’s lateral position x and its signed speed against a pre-defined convention for that road segment (e.g., $x < 0 \Rightarrow$ speed should be positive, and vice versa). A mismatch against this defined sign convention indicates the vehicle is travelling against the permitted direction and is classified as **Wrong-Side Driving**; a consistent sign is classified as **No Anomaly**.

Both branches reuse the same enhanced frame and the same upstream-segmented zone map, so no additional image processing or model inference is required beyond what has already been computed for tracking and zone segmentation.

2 Stage-by-Stage Description

The pipeline takes the pre-processed, enhanced frame I_{enh} as input, together with per-vehicle tracking outputs (centroid, speed) assumed already computed upstream.

2.1 Zone and Sign-Convention Segmentation

No-parking zones are marked in the image ahead of time. The same segmentation pass also marks zones for which a direction convention applies, recording for each such zone whether motion along $+x$ or $-x$ is the legal direction. Formally, the frame is annotated with a zone map

$$Z(x, y) \in \{Z_{\text{NP}}, Z_{\text{dir}}^+, Z_{\text{dir}}^-, Z_{\emptyset}\}$$

where Z_{NP} denotes a no-parking zone, $Z_{\text{dir}}^+/Z_{\text{dir}}^-$ denote zones with a defined positive/negative direction convention, and $Z_{\emptyset}$ is unrestricted road surface.

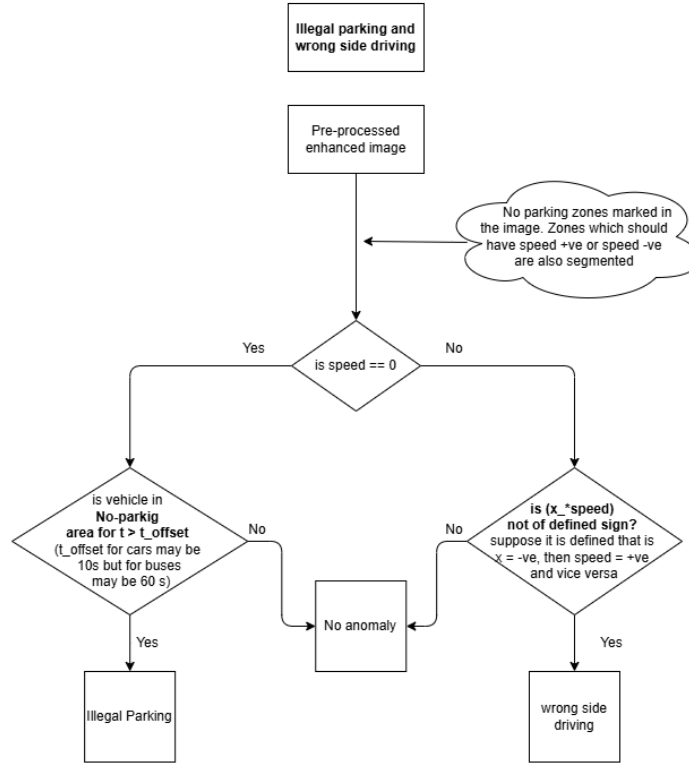


Figure 1: Illegal Parking and Wrong Side Driving

2.2 Motion-State Branching

For a tracked vehicle with centroid (x_c, y_c) and instantaneous speed v , the pipeline first branches on motion state:

$$\text{Branch} = \begin{cases} \text{Parking-check} & v = 0 \\ \text{Direction-check} & v \neq 0 \end{cases}$$

This single test determines which of the two independent anomaly checks is evaluated; a vehicle is never evaluated against both in the same frame.

2.3 Dwell-Time Gated Parking Check

For a stationary vehicle inside a no-parking zone, a per-vehicle dwell timer τ_i accumulates the duration for which the vehicle has remained at $v = 0$ inside Z_{NP} . The class-dependent offset $t_{\text{offset}}(\text{class})$ absorbs brief, non-violating halts, **for example**:

$$t_{\text{offset}}(\text{class}) = \begin{cases} 10 \text{ s} & \text{class} = \text{car} \\ 60 \text{ s} & \text{class} = \text{bus} \end{cases}$$

A vehicle is flagged only once τ_i exceeds this threshold while the vehicle remains inside the zone.

2.4 Sign-Mismatch Direction Check

For a moving vehicle inside a direction-governed zone, the defined convention fixes an expected sign relationship between lateral position x and speed v (e.g., $x < 0 \Rightarrow v > 0$, and vice versa). The product $x_c \cdot v$

is compared against this convention; a product whose sign does not match the defined convention for that zone indicates travel against the permitted direction.

3 Violation Classification Logic

The core decision logic is two independent nested conditionals, evaluated per detected vehicle, per frame:

$$\text{Illegal Parking} = \begin{cases} \text{True} & \text{if } v = 0 \wedge (x_c, y_c) \in Z_{\text{NP}} \wedge \tau_i > t_{\text{offset}}(\text{class}) \\ \text{False} & \text{otherwise} \end{cases}$$

$$\text{Wrong-Side Driving} = \begin{cases} \text{True} & \text{if } v \neq 0 \wedge \text{sign}(x_c \cdot v) \neq \text{sign}_{\text{defined}} \\ \text{False} & \text{otherwise} \end{cases}$$

Interpretation:

- **Illegal Parking:** the vehicle is stationary and located inside a marked no-parking zone, and has remained there beyond the class-appropriate grace period – i.e., it is not merely making a brief, legitimate stop.
- **Wrong-Side Driving:** the vehicle is in motion, and the sign of its lateral-position-speed product contradicts the direction convention defined for that road segment – i.e., it is travelling against permitted traffic flow.

4 Mathematical Formulation

4.1 Combined Decision Function

Let $M = [v \neq 0]$ denote the motion flag, $P = [(x_c, y_c) \in Z_{\text{NP}}]$ denote the no-parking zone membership flag, $T = [\tau_i > t_{\text{offset}}]$ denote the dwell-time-exceeded flag, and $W = [\text{sign}(x_c \cdot v) \neq \text{sign}_{\text{defined}}]$ denote the direction-mismatch flag. The full pipeline reduces to a single composite predicate per tracked vehicle i at frame t :

$$\text{Decision}_i^{(t)} = \begin{cases} \text{Illegal Parking} & M_i^{(t)} = 0, P_i^{(t)} = 1, T_i^{(t)} = 1 \\ \text{No anomaly} & M_i^{(t)} = 0, (P_i^{(t)} = 0 \vee T_i^{(t)} = 0) \\ \text{Wrong-Side Driving} & M_i^{(t)} = 1, W_i^{(t)} = 1 \\ \text{No anomaly} & M_i^{(t)} = 1, W_i^{(t)} = 0 \end{cases}$$

4.2 Boolean / Truth-Table Representation

M	P	T	W	Outcome
0	0	–	–	No anomaly
0	1	0	–	No anomaly
0	1	1	–	Illegal Parking
1	–	–	0	No anomaly
1	–	–	1	Wrong-Side Driving

This is logically equivalent to:

$$\text{Illegal Parking} = \neg M \wedge P \wedge T, \quad \text{Wrong-Side Driving} = M \wedge W$$

5 Algorithmic Summary

Listing 1: Per-frame illegal parking / wrong-side driving classification

```

1  T_OFFSET = {"car": 10.0, "bus": 60.0}    # seconds, class-dependent dwell
    threshold
2  EPSILON = 0.3                            # m/s, stationary threshold (tunable)
3
4  def is_stationary(speed):
5      return abs(speed) <= EPSILON
6
7  def in_no_parking_zone(cx, cy, no_parking_polygons):
8      """Point-in-polygon test against pre-segmented no-parking zones."""
9      return any(point_in_polygon(cx, cy, poly) for poly in no_parking_polygons)
10
11 def direction_mismatch(cx, speed, defined_sign):
12     """
13     defined_sign: +1 if convention is (x<0 => speed>0), -1 if reversed
14     Returns True if sign(x * speed) contradicts the defined convention.
15     """
16     if speed == 0:
17         return False
18     return (cx * speed * defined_sign) < 0
19
20 def classify_vehicle(vehicle, no_parking_polygons, defined_sign, dwell_timers):
21     """
22     vehicle: dict with keys 'id', 'cx', 'cy', 'speed', 'class'
23     dwell_timers: dict {vehicle_id: accumulated_seconds_in_zone}
24     """
25     vid, cx, cy, speed, vclass = (
26         vehicle["id"], vehicle["cx"], vehicle["cy"],
27         vehicle["speed"], vehicle["class"],
28     )
29
30     if is_stationary(speed):
31         if in_no_parking_zone(cx, cy, no_parking_polygons):
32             dwell_timers[vid] = dwell_timers.get(vid, 0.0) + FRAME_DT
33             if dwell_timers[vid] > T_OFFSET.get(vclass, 10.0):
34                 return "ILLEGAL_PARKING"
35         else:
36             dwell_timers[vid] = 0.0
37         return "NO_ANOMALY"
38     else:
39         dwell_timers[vid] = 0.0
40         if direction_mismatch(cx, speed, defined_sign):
41             return "WRONG_SIDE_DRIVING"
42         return "NO_ANOMALY"
43
44
45 def process_frame(tracked_vehicles, no_parking_polygons, defined_sign,
46                 dwell_timers):
47     """
48     tracked_vehicles: list of dicts, each with keys
49         'id', 'cx', 'cy', 'speed', 'class'
50     Returns: dict {vehicle_id: violation_label}
51     """
52     results = {}
53     for v in tracked_vehicles:
54         results[v["id"]] = classify_vehicle(

```

```

54         v, no_parking_polygons, defined_sign, dwell_timers
55     )
56     return results
57
58
59 # --- Example usage ---
60 if __name__ == "__main__":
61     FRAME_DT = 1.0 # seconds per frame (illustrative)
62     no_parking_polygons = [[(100, 400), (260, 400), (260, 520), (100, 520)]]
63     defined_sign = +1 # convention: x < 0 => speed > 0
64     dwell_timers = {}
65
66     tracked_vehicles = [
67         {"id": 1, "cx": 180, "cy": 460, "speed": 0.0, "class": "car"}, #
68         parked in zone
69         {"id": 2, "cx": -50, "cy": 300, "speed": -3.0, "class": "car"}, #
70         against convention
71         {"id": 3, "cx": 600, "cy": 300, "speed": 5.0, "class": "car"}, #
72         normal motion
73     ]
74
75     violations = process_frame(tracked_vehicles, no_parking_polygons,
76                               defined_sign, dwell_timers)
77     for vid, label in violations.items():
78         print(f"Vehicle {vid}: {label}")
79
80     # Output (after dwell exceeds threshold for vehicle 1):
81     # Vehicle 1: ILLEGAL_PARKING
82     # Vehicle 2: WRONG_SIDE_DRIVING
83     # Vehicle 3: NO_ANOMALY

```

6 Important Logic Points

- **No-parking and direction zones are pre-segmented, not inferred per frame.** Both zone types: no-parking regions and zones with a defined direction convention (expected positive or negative sign of motion) are marked once during the upstream segmentation stage, keeping the per-frame decision lightweight.
- **Dwell time, not mere presence, gates the parking violation.** A vehicle momentarily stationary inside a no-parking zone (e.g., passenger boarding) is not flagged; only dwell exceeding t_{offset} triggers *Illegal Parking*.
- **The offset time is class-dependent.** Buses and similar large vehicles are allowed a longer grace period (60s) than cars (10s) to account for routine stops such as boarding.
- **Direction violation is a sign-consistency check, not a magnitude check.** Only the sign of $x_c \cdot v$ relative to the defined convention matters for *Wrong-Side Driving*; the speed magnitude plays no role once motion is confirmed.
- **Thresholding (ϵ) is necessary in practice** to avoid tracker/detection noise being misread as motion, since true $v = 0$ rarely occurs exactly in pixel-derived velocity estimates.
- **Per-vehicle, per-frame evaluation.** The decision is not global to the frame; each tracked vehicle is independently evaluated against both branches' preconditions, allowing simultaneous detection of multiple, distinct violators.